

# Taking Chroma Reranking to the Next Level with a Hybrid Retrieval System

Repost to your network. A new and easy way to share your recommended stories with your followers.

Okay, got it

Sakshi Nepal · read · Apr 1, 2025

👤 207

💬

🔄

🔖

🕒

📄

⋮

## Why Settle for Mediocre Search? Let's Fix It.

Search engines suck when they don't return what you actually need. ChromaDB is great for fast vector search, but it struggles when precision matters. That's why I built a **Hybrid Reranking System** — a beast of a pipeline that fuses **BM25**, **Vector Similarity**, **ColBERT Reranking**, and **Contextual Matching** to deliver next-level search accuracy.

In this article, I'll break down why this matters, how it works, and why you need it if you're building Retrieval-Augmented Generation (RAG) systems that don't just retrieve but actually **understand what matters**.

## Why ChromaDB Over Qdrant? Because Precision Matters

ChromaDB and Qdrant are both vector databases, but ChromaDB has some distinct advantages for hybrid retrieval and reranking:

### 1. ChromaDB Has Better Reranking Support

Qdrant focuses primarily on vector search, but it lacks built-in flexible reranking options. ChromaDB, on the other hand, can seamlessly integrate BM25, ColBERT, and hybrid retrieval strategies — making it the superior choice for multi-modal ranking.

### 2. Faster Hybrid Search Without Complexity

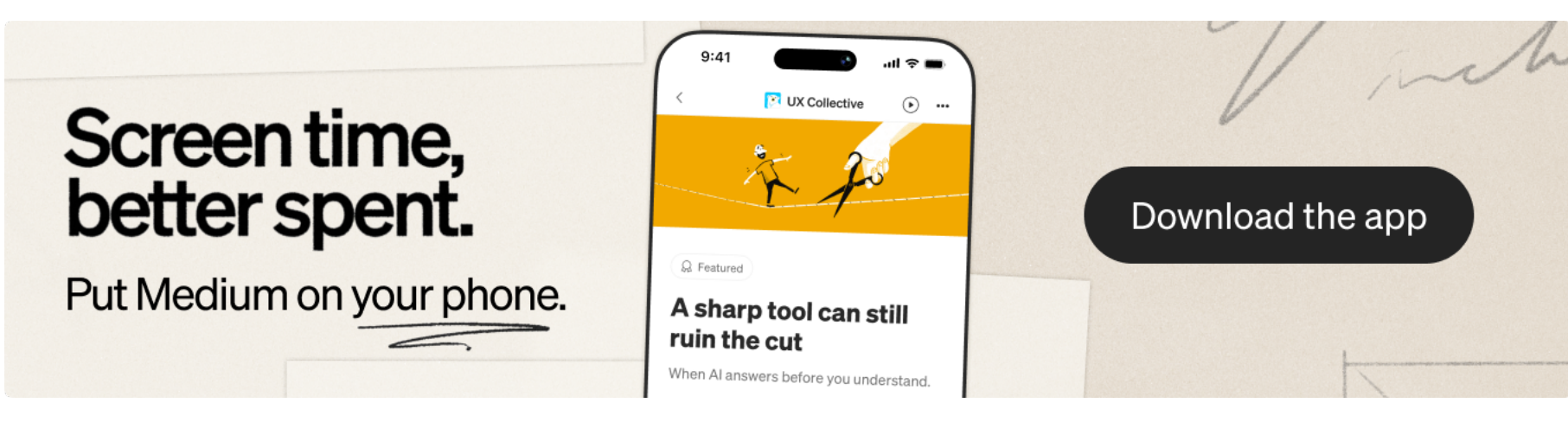
Qdrant requires manual workarounds to mix BM25 and embeddings, while ChromaDB natively supports this hybrid approach. Less setup, fewer headaches, better results.

### 3. Scalability & Integration

- ChromaDB has low-latency retrieval that scales efficiently
- It integrates easily into RAG pipelines without needing extra search layers
- Unlike Qdrant, it's optimized for multi-collection search, making it better for complex retrieval tasks

### 4. Qdrant Struggles With Large Datasets

If you're dealing with millions of documents, Qdrant's performance can degrade due to index fragmentation issues. ChromaDB, however, is designed to handle massive datasets efficiently.



Bottom line? ChromaDB gives you speed, flexibility, and better ranking — out of the box.

## Why Build This? Because Vanilla ChromaDB Isn't Enough

### 1. ChromaDB Drops the Ball on Reranking

ChromaDB retrieves documents using embeddings, but ranking? Not so much. That means:

- It struggles with ambiguous queries
- It returns garbage results when the dataset is messy
- It fails when the best match isn't purely based on semantics

### 2. BM25 and Vector Search Alone Can't Cut It

Each method has fatal flaws:

- BM25 nails exact keyword matches but flops on synonyms
- Vector search gets semantics, but sucks at long docs & precise ranking

The fix? **ColBERT reranking**, which adds deep contextual relevance to separate “meh” results from golden hits.

### 3. RAG Pipelines Live or Die by Good Retrieval

If your retrieval system is mediocre, your LLM will generate **nonsense**. This hybrid approach ensures only the most relevant, context-aware documents make it in — boosting LLM accuracy and cutting token costs.

## How It Works: The Multi-Stage Reranking Gauntlet

### 1. Query Preprocessing: Clean Up the Mess

Before we even retrieve anything, we sanitize the query to avoid dumb mismatches:

```
import re

def _preprocess_query(query: str) -> str:
    query = query.strip().lower()
    query = re.sub(r'[\s\w\s?!]', ' ', query) # Strip special characters
    return " ".join(query.split()) # Remove extra spaces
```

### 2. Hybrid Retrieval: BM25 + Vector Search = 🔥

#### Vector Search via ChromaDB:

```
async def _retrieve_vector(self, query: str) -> List[Document]:
    query_embedding = self._embeddings.embed_query(query)
    results = self._client.get_or_create_collection(self._collection_name).query(
        query_embeddings=query_embedding,
        n_results=self._top_k * 2,
        include=["documents", "metadatas", "distances"]
    )
    return [Document(page_content=doc, metadata={"vector_score": max(0, 1 - dist)})
            for doc, dist in zip(results["documents"][0], results["distances"][0])]
```

#### BM25 Retrieval:

```
async def _retrieve_bm25(self, query: str) -> List[Document]:
    bm25_retriever = BM25Retriever.from_documents(all_docs)
    bm25_retriever.k = self._top_k * 2
    return bm25_retriever.invoke(query)
```

### 3. ColBERT Reranking: Deep Contextual Boost

```
def _colbert_rerank(self, query: str, docs: List[Document]) -> List[Document]:
    reranked = self._colbert_reranker.rerank(query, [doc.page_content for doc in docs])
    for i, result in enumerate(reranked):
        docs[result["result_index"]].metadata["colbert_score"] = result["score"]
    return docs
```

### 4. Contextual Matching: Term Overlap Matters

```
def _contextual_similarity(self, query: str, docs: List[Document]) -> List[Document]:
    query_terms = set(self._preprocess_query(query).split())
    for doc in docs:
        doc_terms = set(doc.page_content.lower().split())
        overlap = len(query_terms & doc_terms) / max(len(query_terms), 1)
        doc.metadata["context_score"] = overlap
    return docs
```

### 5. Normalization & Fusion: The Power Play

Normalize all scores so nothing dominates unfairly:

```
def _normalize_scores(self, docs: List[Document]) -> List[Document]:
    for method in ["vector", "bm25", "colbert", "context"]:
        scores = [doc.metadata.get(f"{method}_score", 0) for doc in docs]
        if max(scores) > min(scores):
            normalized = (scores - min(scores)) / (max(scores) - min(scores) + 1)
            for doc, norm_score in zip(docs, normalized):
                doc.metadata[f"{method}_score"] = norm_score
    return docs
```

Combine scores with adaptive weighting:

```
def _advanced_fusion(self, docs: List[Document]) -> List[Document]:
    for doc in docs:
        doc.metadata["final_score"] = sum(
            self._default_weights[m] * doc.metadata.get(f"{m}_score", 0)
            for m in self._default_weights
        )
    return sorted(docs, key=lambda x: x.metadata["final_score"], reverse=True)[:self._top_k]
```

## GITHUB LINK:

GitHub - sakshitx/re-ranking

Contribute to sakshitx/re-ranking development by creating an account on GitHub.

github.com

👤 207

💬

🔄

🔖

🕒

📄

⋮

Written by Sakshi Nepal

4 followers · 1 following

Always curious

Follow

## No responses yet

Bhalchandra Madhekar

What are your thoughts?

